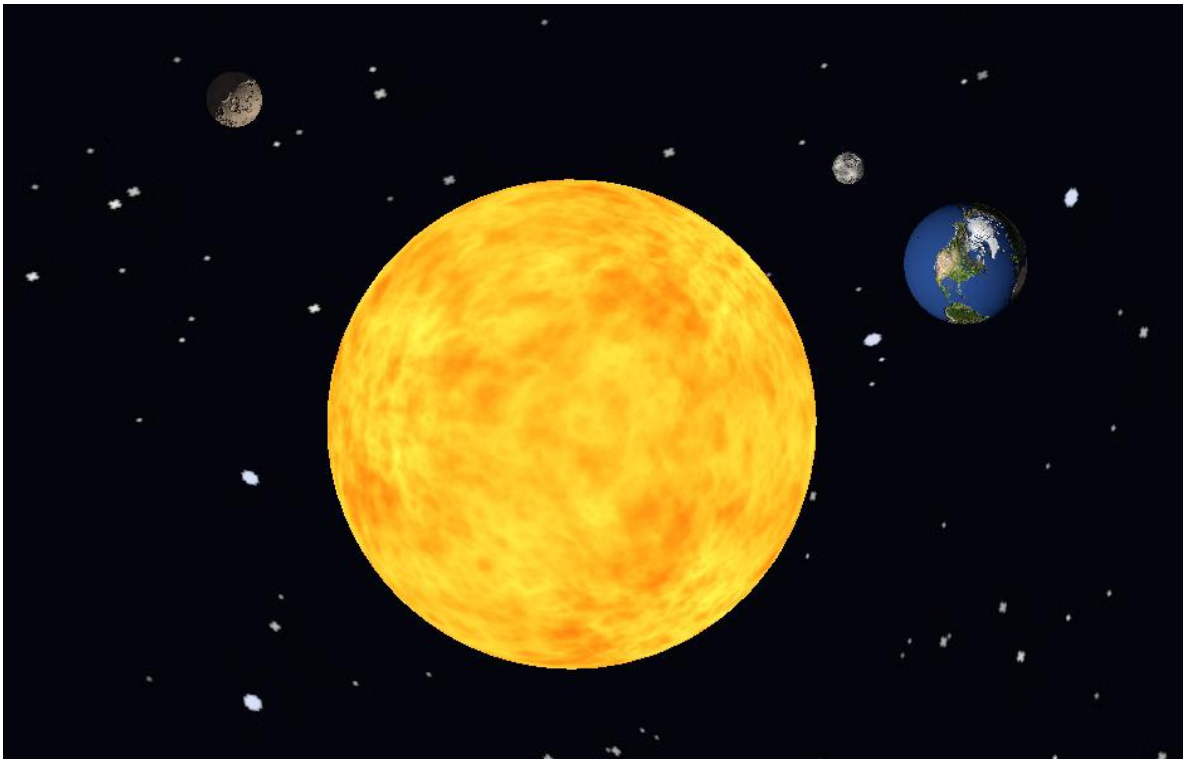
Relatório Projeto 2

INF1761

Lucas Brito – 2411907



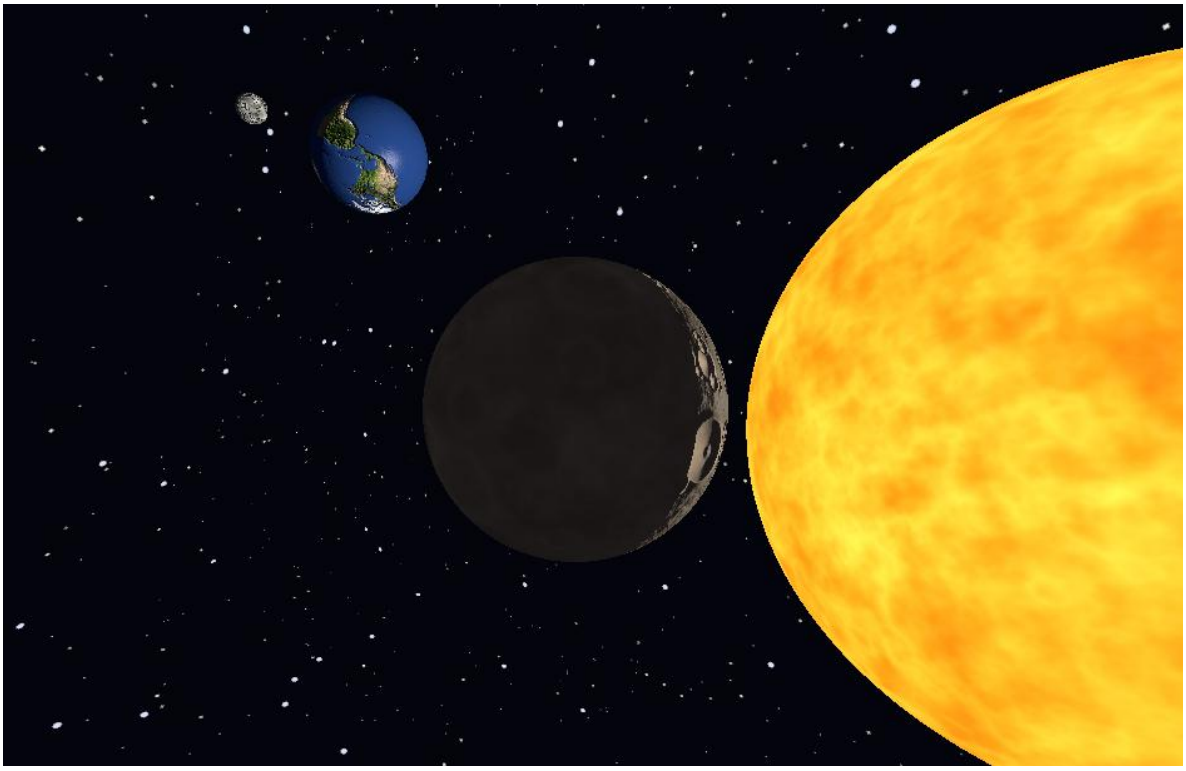
O projeto 2 (opção 2) nos pedia para criar uma animação 3D de um mini-sistema solar com Sol, Mercúrio (ou Vênus), Terra e Lua, usando o framework de grafo de cena. A aplicação básica pedia iluminação por fragmento com uma fonte de luz na posição do Sol, a translação da Terra em torno do Sol e sua rotação em torno do próprio eixo, a translação da Lua em torno da Terra, um astro adicional entre o Sol e a Terra com translação em torno do Sol, texturas nos astros e uma textura de fundo representando o espaço. Além disso, implementei as três extensões propostas: o mapeamento de rugosidade (bump mapping), a troca de câmeras com uma engine, e o plano de corte. Como o projeto foi feito individualmente, não há divisão de tarefas.

Para organizar o desenvolvimento, dividi o trabalho em 6 etapas:

A Cena 3D e as Texturas: Todos os astros são instâncias da mesma esfera do framework (Sphere), apenas com escalas diferentes no grafo de cena. O Sol fica na origem e Mercúrio foi o astro adicional escolhido, em uma órbita interna entre o Sol e a Terra. Para a cor de cada astro usei texturas decal em rgba8unorm-srgb: a Terra usa o mapa de continentes e oceanos que veio no framework, e por isso sua rotação em torno do próprio eixo fica visível. As texturas do Sol, da Lua e de Mercúrio eram desenhos 2D feitos para os discos do Projeto 1 e apareciam com emendas e distorções na esfera, então escrevi um script em Python (tools/make_solar_textures.py) que gera texturas equirretangulares 2:1 sem costura, amostrando um ruído do próprio framework diretamente em coordenadas 3D da esfera. Para o fundo usei a SkyBox do framework: um cubo de 6 faces carregado pela classe TexCube a partir de uma imagem em cruz (space_cross.png, montada por outro script a partir do space.jpg), recentrado na posição da câmera a cada quadro pelo SkyBoxTransform. Na pipeline do fundo desliguei a escrita no depth buffer e forcei z = w no vertex shader, então o cubo fica sempre atrás de tudo, como um fundo infinito.



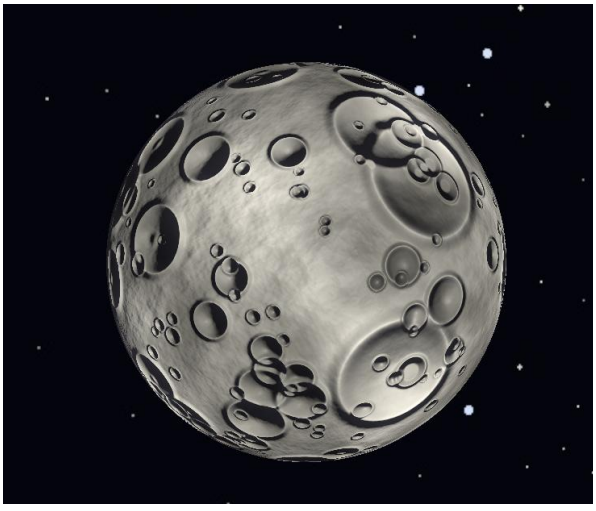
A Iluminação da Cena: Posicionei uma luz pontual (PointLight) na origem do mundo, exatamente no centro do Sol. O Sol usa um shader próprio e emissivo (sun.wgsl), que só mostra sua textura, já que ele é a própria fonte de luz. A Terra, a Lua e Mercúrio compartilham o shader planet.wgsl, que calcula a iluminação de Blinn-Phong por fragmento no espaço da câmera: o vertex shader entrega os vetores de luz e de visão, e o fragment shader os renormaliza pixel a pixel e soma as componentes ambiente, difusa e especular multiplicadas pela cor da textura. Com isso só o hemisfério voltado para o Sol é iluminado, o terminador entre dia e noite aparece em cada planeta e a Lua exibe suas fases.



O Movimento dos Astros: Separei no grafo de cena os nós de pivô (rotação), de posição (translação) e de corpo (escala). Para a Terra, o nó de órbita gira em torno do Sol e tem como filho o nó de posição, que só faz a translação até o raio orbital. Desse nó de posição saem dois ramos irmãos: um com a inclinação do eixo (23,4°), o pivô de rotação própria e a esfera da Terra, e outro com o pivô da órbita da Lua (inclinada em 5° em relação ao plano da órbita da Terra), a translação até a distância Terra–Lua e a esfera da Lua. Assim a Lua herda o movimento orbital da Terra pela pilha de matrizes, mas, por ser irmã e não filha do nó de rotação, não gira junto com os

continentes. Mercúrio tem seu próprio pivô de translação em torno do Sol e uma rotação própria, e a Lua gira na mesma taxa da sua órbita para mostrar sempre a mesma face à Terra. Tudo é atualizado a cada quadro pela classe SolarEngine, derivada de Engine, que acumula as rotações proporcionalmente ao tempo dt, com teclas para pausar (P) e alterar a velocidade (+ e -).

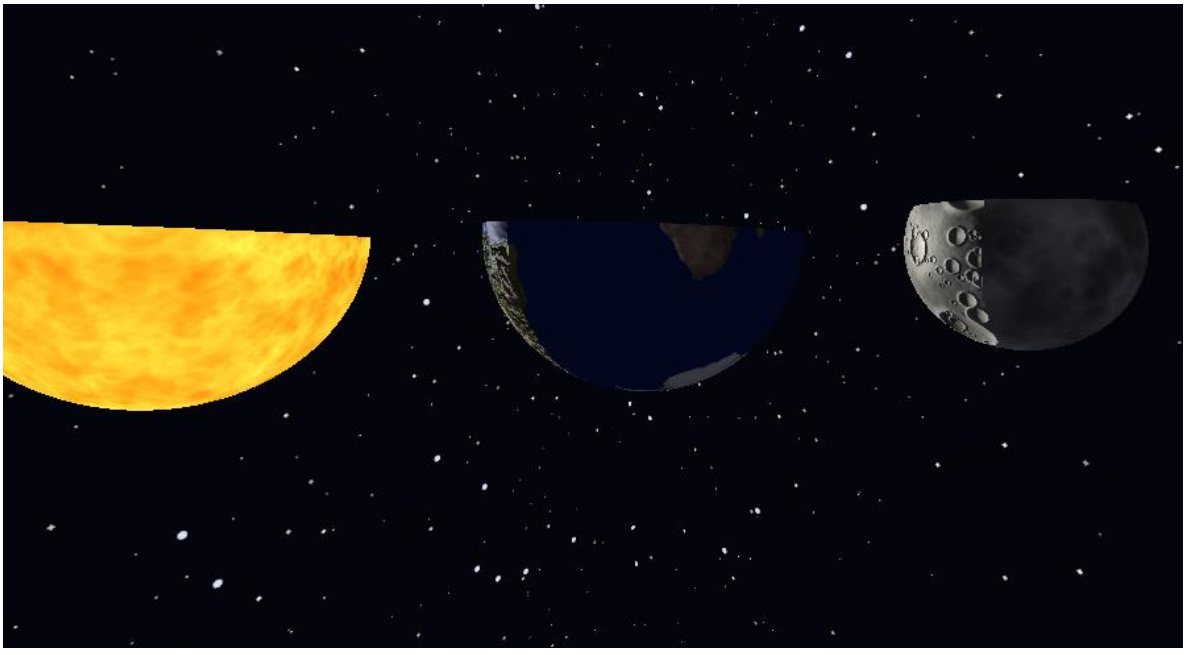
O Mapeamento de Rugosidade: Segui o método da aula, com mapa de normais no espaço tangente. Primeiro estendi a classe Texture com um parâmetro opcional de formato (única alteração que fiz no código do framework), para carregar os mapas de normais em rgba8unorm e evitar a curva de sRGB, que deslocaria a normal em repouso (0.5, 0.5, 1.0). A Terra usa o earth-normal.png do framework. Para a Lua e Mercúrio, o script gera um campo de elevação com crateras de perfil côncavo e borda elevada, e outro script (tools/make_normal_map.py) o converte em mapa de normais pelo produto vetorial das diferenças finitas, como no exercício da aula. No vertex shader monto a base (t, b, n) por vértice com o atributo de tangente da esfera e levo a luz e a visão para o espaço tangente. No fragment shader decodifico a normal por n = 2c − 1 e aplico o teste da normal macro (select(0.0, dot(normal, light), light.z > 0.0)), para que o relevo na face voltada contra o Sol não receba luz. Um uniforme (bump_amount) escala a inclinação da normal: a tecla B liga e desliga o relevo e as teclas [e] ajustam sua profundidade, o que deixa as crateras bem visíveis ao aproximar a câmera.





A Troca de Câmeras: A tecla C alterna entre quatro câmeras (também acessíveis pelas teclas 1 a 4). A primeira é a câmera global, com manipulador arcball para orbitar todo o sistema. A segunda é a câmera pedida no enunciado: a classe EarthCameraEngine, derivada de Engine, lê a cada quadro as matrizes de modelo da Terra e da Lua no grafo (GetModelMatrix), posiciona o olho logo acima da superfície da Terra, na direção da Lua, e aponta o centro para a Lua. Assim a Lua fica sempre no centro da tela, e como ela é iluminada pelo Sol é possível observar suas mudanças de fase e o relevo das crateras no terminador. Acrescentei ainda duas câmeras de inspeção, uma na Terra e outra em Mercúrio, ligadas aos nós de posição de cada astro (SetReference) e com arcball próprio. A roda do mouse faz zoom em qualquer câmera, o que facilita observar o relevo de perto.

O Plano de Corte: A tecla X ativa um plano de corte usando a classe ClipPlane do framework. O plano é o do mundo y = 0 (o plano das órbitas), transformado a cada quadro para o espaço da câmera ativa pela transposta da inversa da matriz de visão. No fragment shader, os fragmentos com $n \cdot p + d < 0$ são descartados com discard. Como a esfera é só uma casca de triângulos, desativei o culling e uso o front_facing para pintar a face interna com uma cor lisa de seção, mostrando que o astro é maciço. Como o Sol, Mercúrio e a Terra orbitam no plano y = 0, o corte os divide exatamente ao meio. Já a órbita da Lua é inclinada em 5°, então ela atravessa o plano de um lado para o outro e a fatia do corte varia durante a animação.



Controles: C ou 1 a 4 trocam a câmera; arrastar o mouse gira o arcball; a roda do mouse dá zoom; B liga e desliga o relevo; [e] ajustam sua profundidade; X liga e desliga o plano de corte; P pausa; + e - alteram a velocidade; Q ou Esc fecham a janela.

Link do vídeo: https://drive.google.com/file/d/1v075Z8ZvPRgqxNbAlomCjkICitaengqQ/view?usp=sharing

Link do repositório: https://github.com/LucasBrito06/inf1761_Projeto2